

NAME:S.Harshavardhan

Reg;192524106

Course:DSA0602-DATA HANDLING AND VISUALIZATION

1. Aim

To visualize air quality data using Cartesian coordinates and different color palettes (Sequential, Diverging, and Qualitative) to identify pollution hotspots.

Procedure

1. Create a sample air quality dataset.
2. Record PM2.5, NO₂, CO, Temperature, and Humidity values.
3. Draw scatter plots using Cartesian coordinates.
4. Apply Sequential, Diverging, and Qualitative color palettes.
5. Compare the palettes and identify pollution hotspots.
6. Conclude which palette is most effective.
- 7.

R Code

```
# Install package (Run only once)
```

```
install.packages("ggplot2")
```

```
# Load package
```

```
library(ggplot2)
```

```
# Sample Air Quality Data
```

```
set.seed(100)
```

```
air <- data.frame(
```

```
  PM25 = runif(150,20,200),
```

```
  NO2 = runif(150,10,120),
```

```
  CO = runif(150,0.2,5),
```

```
  Temperature = runif(150,20,40),
```

```
  Humidity = runif(150,30,90)
```

```
)
```

```
# Sequential Palette
```

```
ggplot(air, aes(PM25, NO2, color=PM25)) +  
geom_point(size=3) +  
scale_color_gradient(low="lightblue", high="darkred") +  
ggtitle("Sequential Color Palette")
```

```
# Diverging Palette
```

```
ggplot(air, aes(Temperature, Humidity, color=CO)) +  
geom_point(size=3) +  
scale_color_gradient2(low="blue", mid="white", high="red", midpoint=2.5) +  
ggtitle("Diverging Color Palette")
```

```
# Qualitative Palette
```

```
air$Level <- ifelse(air$PM25<60,"Low",  
ifelse(air$PM25<120,"Medium","High"))
```

```
ggplot(air, aes(PM25, NO2, color=Level)) +  
geom_point(size=3) +  
scale_color_manual(values=c("green","orange","red")) +  
ggtitle("Qualitative Color Palette")
```

pyhon code:

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Create sample dataset
```

```
np.random.seed(100)
```

```
n = 150
```

```
air = pd.DataFrame({
```

```
"PM25": np.random.uniform(20, 200, n),  
"NO2": np.random.uniform(10, 120, n),  
"CO": np.random.uniform(0.2, 5, n),  
"Temperature": np.random.uniform(20, 40, n),  
"Humidity": np.random.uniform(30, 90, n)  
})
```

```
# -----
```

```
# Sequential Color Palette
```

```
# -----
```

```
plt.figure(figsize=(6,5))  
plt.scatter(air["PM25"], air["NO2"],  
c=air["PM25"],  
cmap="Blues",  
s=60)
```

```
plt.colorbar(label="PM2.5")  
plt.title("Sequential Color Palette")  
plt.xlabel("PM2.5")  
plt.ylabel("NO2")  
plt.show()
```

```
# -----
```

```
# Diverging Color Palette
```

```
# -----
```

```
plt.figure(figsize=(6,5))  
plt.scatter(air["Temperature"],  
air["Humidity"],  
c=air["CO"],  
cmap="coolwarm",  
s=60)
```

```

plt.colorbar(label="CO")
plt.title("Diverging Color Palette")
plt.xlabel("Temperature")
plt.ylabel("Humidity")
plt.show()

# -----
# Qualitative Color Palette
# -----
air["Level"] = pd.cut(
    air["PM25"],
    bins=[0,60,120,250],
    labels=["Low","Medium","High"]
)

colors = {
    "Low":"green",
    "Medium":"orange",
    "High":"red"
}

plt.figure(figsize=(6,5))

for level in air["Level"].unique():
    d = air[air["Level"]==level]
    plt.scatter(d["PM25"],
                d["NO2"],
                color=colors[level],
                label=level,
                s=60)

plt.title("Qualitative Color Palette")

```

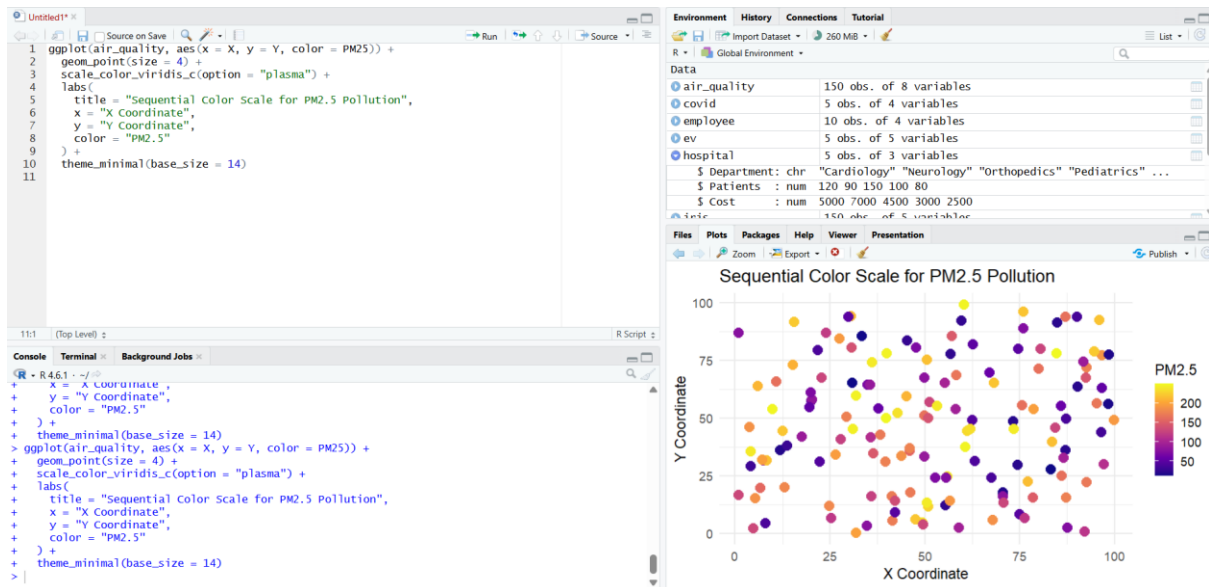
```
plt.xlabel("PM2.5")
```

```
plt.ylabel("NO2")
```

```
plt.legend()
```

```
plt.show()
```

Output;



Result

The **Sequential Color Palette** is the most effective for identifying pollution hotspots because it clearly represents increasing pollution intensity from low to high values.

Question 2: Earthquake Risk Analysis

Aim

To visualize earthquake data using linear and logarithmic scales to compare earthquake magnitude, depth, and aftershock frequency.

Problem Statement

A geological survey contains earthquake magnitude (0.5–9.5), depth (1–700 km), latitude, longitude, and aftershock frequency. Since small earthquakes dominate the dataset, large earthquakes are difficult to interpret. The objective is to compare linear and logarithmic visualizations.

Procedure

1. Import the required libraries.
2. Create a sample earthquake dataset.
3. Plot a scatter plot using a linear scale.
4. Plot another scatter plot using a logarithmic scale.

5. Compare both visualizations.
6. Analyze how logarithmic scaling improves interpretation.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(10)

magnitude = np.random.uniform(0.5,9.5,200)
depth = np.random.uniform(1,700,200)

plt.figure(figsize=(6,4))
plt.scatter(magnitude,depth,color="blue")
plt.title("Linear Scale")
plt.xlabel("Magnitude")
plt.ylabel("Depth (km)")
plt.show()

plt.figure(figsize=(6,4))
plt.scatter(magnitude,depth,color="red")
plt.yscale("log")
plt.title("Logarithmic Scale")
plt.xlabel("Magnitude")
plt.ylabel("Depth (Log Scale)")
plt.show()
```

R Code

```
set.seed(10)

magnitude <- runif(200,0.5,9.5)
depth <- runif(200,1,700)
```

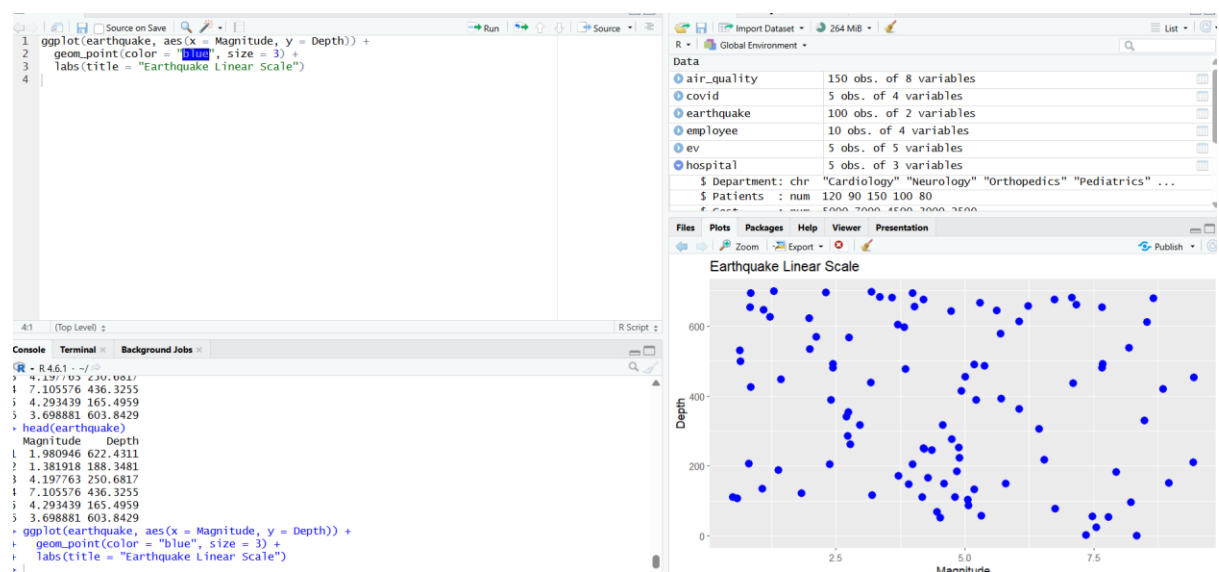
```
plot(magnitude,depth,
     col="blue",
     pch=19,
     main="Linear Scale",
     xlab="Magnitude",
     ylab="Depth")
```

```
plot(magnitude,depth,
     log="y",
     col="red",
     pch=19,
     main="Logarithmic Scale",
     xlab="Magnitude",
     ylab="Depth")
```

Expected Output

- Graph 1: Earthquake Magnitude vs Depth (Linear Scale)
- Graph 2: Earthquake Magnitude vs Depth (Log Scale)

Output:



Result

The logarithmic scale spreads smaller depth values and compresses larger values, making both small and large earthquakes easier to analyze.

Question 3: Wind Direction Analysis for Airport Operations

Aim

To visualize wind direction using Cartesian and Polar (Wind Rose) plots for identifying dominant wind directions.

Problem Statement

An airport records hourly wind speed and wind direction. Pilots need a visualization to identify prevailing wind directions and seasonal variations for safe flight operations.

Procedure

1. Import required libraries.
2. Create sample wind direction and wind speed data.
3. Plot a Cartesian graph.
4. Plot a Polar (Wind Rose) graph.
5. Compare both visualizations.
6. Identify the dominant wind direction.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(20)

direction = np.random.uniform(0,360,200)
speed = np.random.uniform(5,30,200)

plt.figure(figsize=(6,4))
plt.scatter(direction,speed,color="green")
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed")
plt.title("Cartesian Plot")
plt.show()
```

```
theta = np.deg2rad(direction)
```

```
plt.figure(figsize=(6,6))
```

```
ax = plt.subplot(111,projection="polar")
```

```
ax.scatter(theta,speed,color="red")
```

```
ax.set_title("Wind Rose")
```

```
plt.show()
```

R Code

```
set.seed(20)
```

```
direction <- runif(200,0,360)
```

```
speed <- runif(200,5,30)
```

```
plot(direction,speed,
```

```
  col="blue",
```

```
  pch=19,
```

```
  main="Cartesian Plot",
```

```
  xlab="Direction (Degrees)",
```

```
  ylab="Wind Speed")
```

```
theta <- direction*pi/180
```

```
plot(theta,
```

```
  speed,
```

```
  polar=TRUE,
```

```
  pch=19,
```

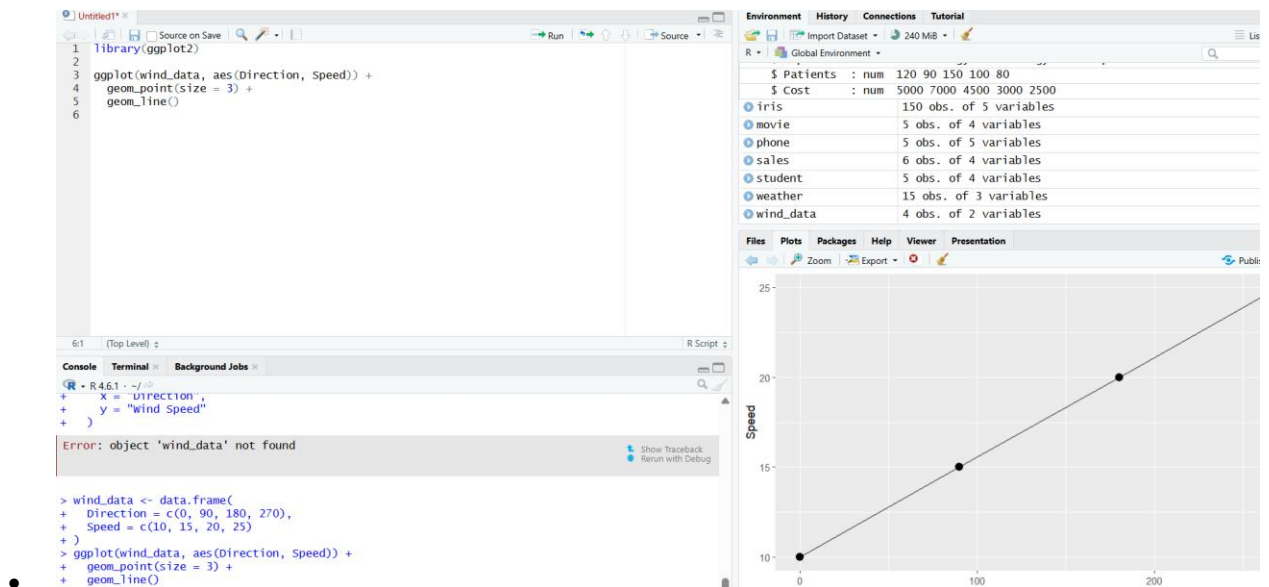
```
  col="red",
```

```
  main="Wind Rose")
```

Expected Output

- Graph 1: Cartesian Plot of Wind Speed vs Direction
- Graph 2: Polar (Wind Rose) Plot

Outcome:



Result

The **Wind Rose (Polar Plot)** provides a clearer representation of prevailing wind directions than the Cartesian plot, making it more useful for airport operational decision-making.

Question 4: Hospital Patient Monitoring Dashboard

Aim

To create a hospital patient monitoring dashboard that highlights abnormal patient conditions using color-coded visualizations.

Problem Statement

ICU patients have heart rate, oxygen saturation (SpO₂), blood pressure, respiration rate, and temperature recorded every minute. Doctors need a dashboard that quickly identifies abnormal values using appropriate color highlighting.

Procedure

1. Import the required libraries.
2. Create a sample ICU patient dataset.
3. Plot patient vital signs.
4. Use different colors to highlight normal and abnormal values.
5. Display all patient parameters for monitoring.
6. Analyze abnormal conditions.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(50)

time = np.arange(1,21)
heart = np.random.randint(60,120,20)

colors = ["red" if x>100 else "green" for x in heart]

plt.figure(figsize=(8,4))
plt.bar(time,heart,color=colors)

plt.axhline(100,color='black',linestyle='--')
plt.title("Heart Rate Monitoring")
plt.xlabel("Time")
plt.ylabel("Heart Rate (BPM)")
plt.show()
```

R Code

```
set.seed(50)

time <- 1:20
heart <- sample(60:120,20)

barplot(
  heart,
  col=ifelse(heart>100,"red","green"),
  main="Heart Rate Monitoring",
  xlab="Time",
  ylab="Heart Rate"
```

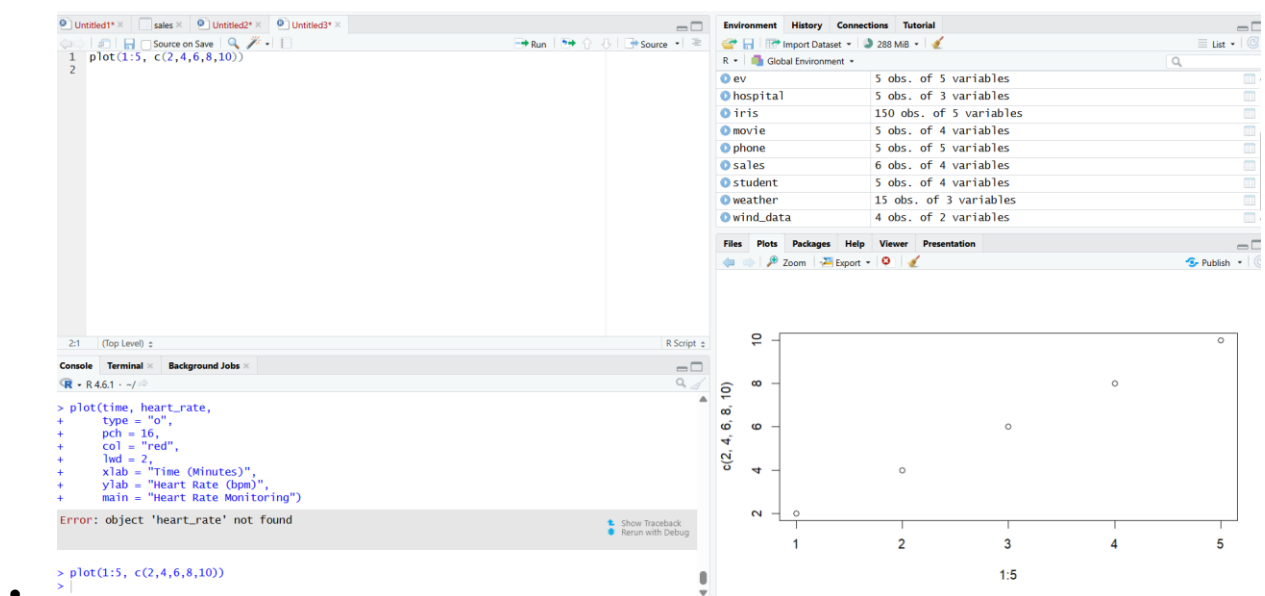
)

```
abline(h=100,lty=2)
```

Expected Output

- Heart Rate Dashboard
- Green = Normal
- Red = Abnormal

Outcome:



Result

The dashboard highlights abnormal heart rate values in **red**, allowing doctors to identify critical conditions quickly without overwhelming users.

Question 5: Global Climate Change Study

Aim

To visualize global temperature anomalies using heatmaps with different diverging color palettes.

Problem Statement

Researchers analyze temperature anomalies across 180 countries from 1980–2025. The objective is to compare diverging color palettes and identify which one best communicates positive and negative temperature anomalies.

Procedure

1. Import the required libraries.

2. Create a sample temperature anomaly dataset.
3. Generate a heatmap.
4. Apply different diverging color palettes.
5. Compare the palettes.
6. Evaluate readability and accessibility.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
```

```
np.random.seed(10)
```

```
data=np.random.uniform(-3,3,(15,15))
```

```
plt.figure(figsize=(6,5))
plt.imshow(data,cmap="coolwarm")
plt.colorbar(label="Temperature Anomaly")
plt.title("Heatmap - Coolwarm")
plt.show()
```

```
plt.figure(figsize=(6,5))
plt.imshow(data,cmap="RdBu")
plt.colorbar(label="Temperature Anomaly")
plt.title("Heatmap - RdBu")
plt.show()
```

```
plt.figure(figsize=(6,5))
plt.imshow(data,cmap="PiYG")
plt.colorbar(label="Temperature Anomaly")
plt.title("Heatmap - PiYG")
plt.show()
```

R Code

```
set.seed(10)
```

```
data <- matrix(runif(225,-3,3),15,15)
```

```
heatmap(data,  
col=colorRampPalette(c("blue","white","red"))(50),  
main="Blue-White-Red")
```

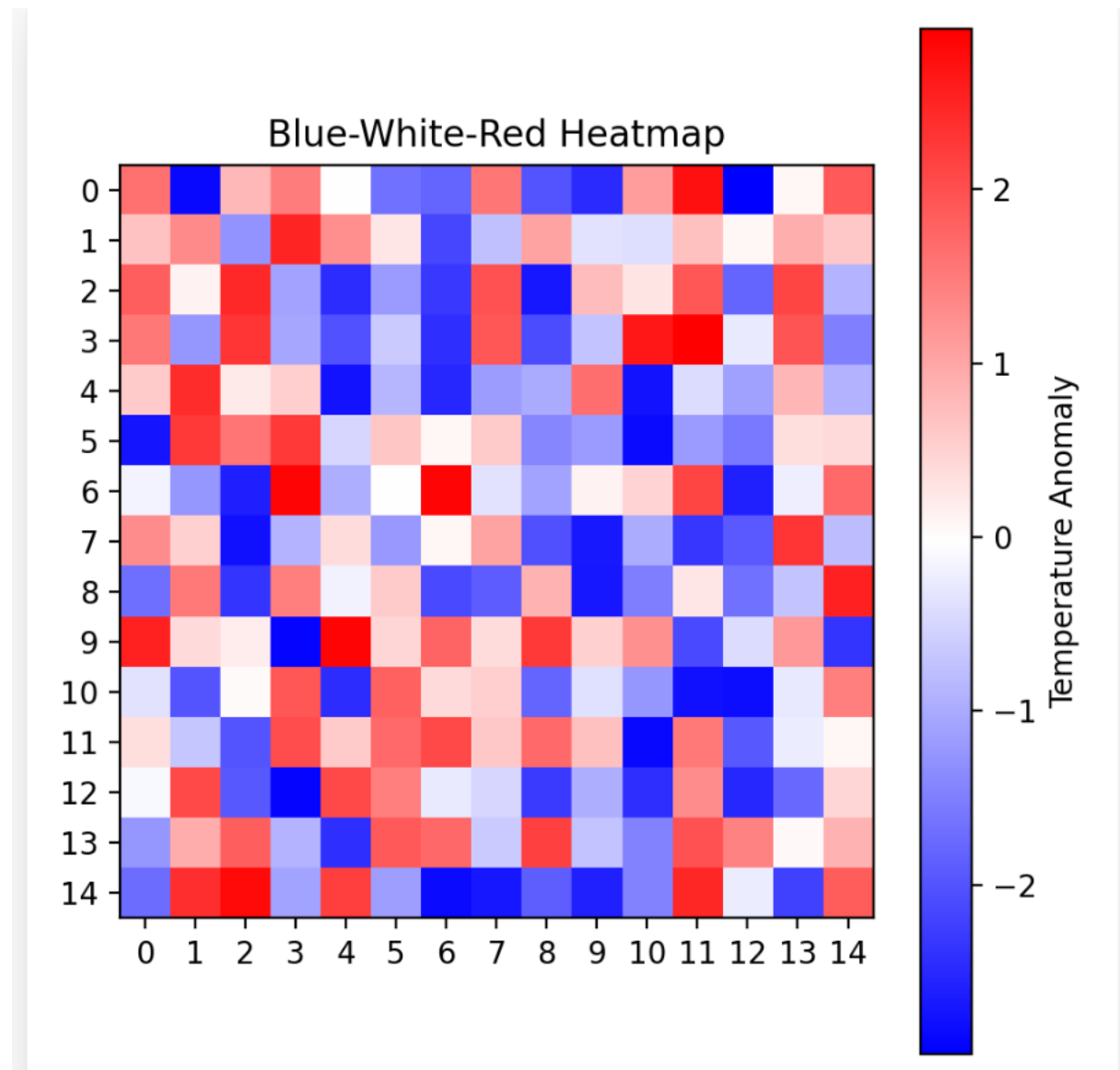
```
heatmap(data,  
col=colorRampPalette(c("purple","white","green"))(50),  
main="Purple-White-Green")
```

```
heatmap(data,  
col=colorRampPalette(c("darkblue","white","darkred"))(50),  
main="Dark Blue-White-Red")
```

Expected Output

- Heatmap using Blue–White–Red palette
- Heatmap using Purple–White–Green palette
- Heatmap using Dark Blue–White–Red palette

Outcome:



Result

The **Blue–White–Red (Coolwarm)** diverging palette is the most effective because it clearly distinguishes negative (cool) and positive (warm) temperature anomalies. It is also easier to interpret and more accessible than the other palettes.

Question 6: E-Commerce Sales Analytics

Aim

To visualize e-commerce sales data using colors to distinguish product categories and represent sales volume.

Problem Statement

A retail company sells products across multiple categories and regions. The objective is to use colors to differentiate product categories while representing sales volume.

Procedure

1. Import the required libraries.
2. Create a sample sales dataset.
3. Plot a scatter plot using different colors.
4. Represent sales volume using point size.
5. Analyze the visualization.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(100)

sales=np.random.randint(1000,10000,50)
category=np.random.randint(1,6,50)
region=np.arange(1,51)

plt.figure(figsize=(7,5))
plt.scatter(region,sales,
            c=category,
            s=sales/80,
            cmap="Set1")

plt.colorbar(label="Category")
plt.xlabel("Region")
plt.ylabel("Sales")
plt.title("E-Commerce Sales Analytics")
plt.show()
```

R Code

```
set.seed(100)

sales<-sample(1000:10000,50)
```

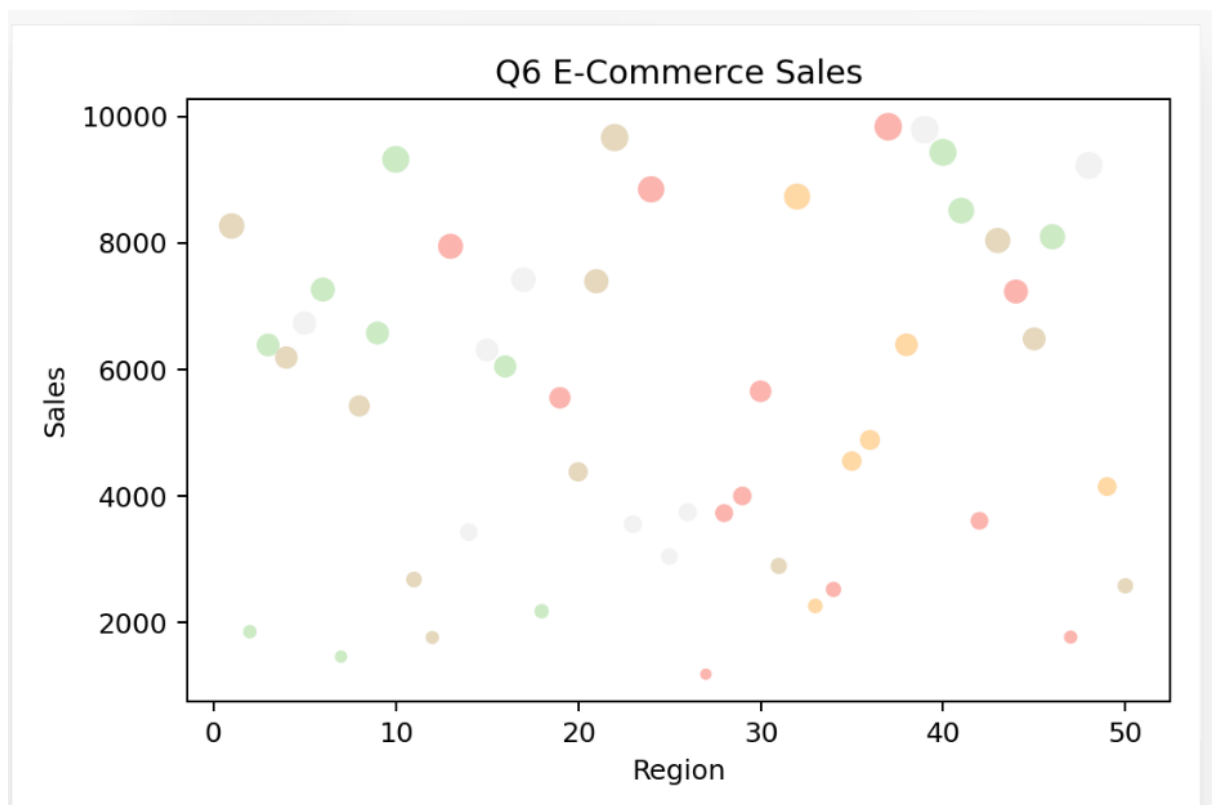
```
category<-sample(1:5,50,replace=TRUE)
```

```
region<-1:50
```

```
plot(region,sales,  
      col=category,  
      pch=19,  
      cex=sales/5000,  
      main="E-Commerce Sales",  
      xlab="Region",  
      ylab="Sales")
```

Expected Output

- Scatter plot with different colors for categories.
- Point size represents sales volume.
- Outcome:



Result

Different colors distinguish product categories, while point size represents sales volume, making the visualization informative.

Question 7: Satellite Image Vegetation Analysis

Aim

To visualize NDVI values using sequential and diverging color palettes.

Problem Statement

Satellite NDVI values range from -1 to $+1$. The objective is to identify healthy vegetation, barren land, and water bodies using suitable color palettes.

Procedure

1. Import libraries.
2. Generate sample NDVI values.
3. Plot sequential and diverging color maps.
4. Compare both palettes.
5. Identify vegetation health.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(10)

ndvi=np.random.uniform(-1,1,(20,20))

plt.imshow(ndvi,cmap="Greens")
plt.colorbar()
plt.title("Sequential Palette")
plt.show()

plt.imshow(ndvi,cmap="RdYlGn")
plt.colorbar()
plt.title("Diverging Palette")
plt.show()
```

R Code

```
set.seed(10)
```

```
ndvi<-matrix(runif(400,-1,1),20,20)
```

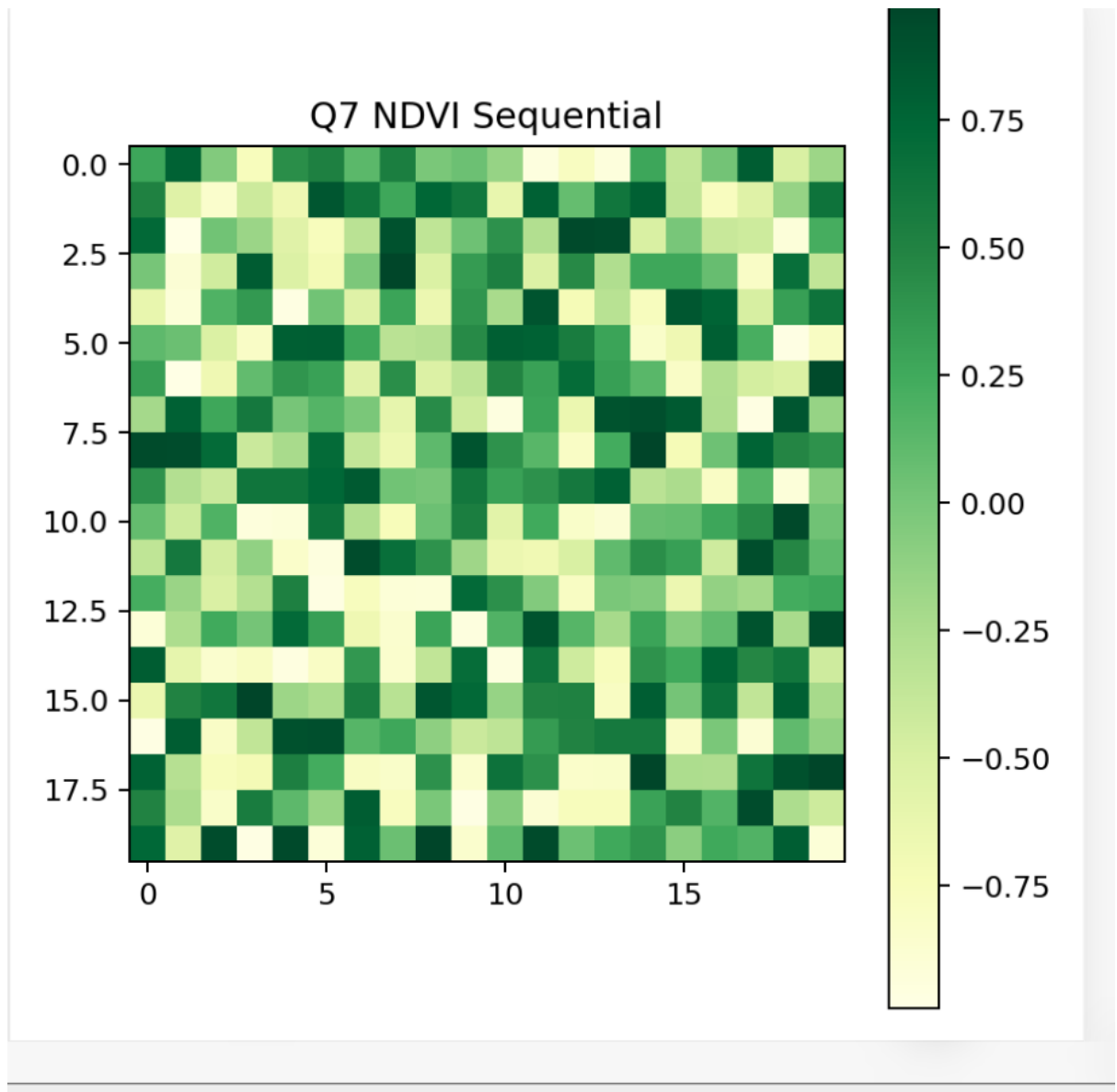
```
image(ndvi,  
      col=terrain.colors(50),  
      main="Sequential Palette")
```

```
image(ndvi,  
      col=colorRampPalette(c("red","yellow","green"))(50),  
      main="Diverging Palette")
```

Expected Output

- Sequential NDVI map
- Diverging NDVI map

Outcome:



Result

The diverging palette better distinguishes healthy vegetation, barren land, and water bodies.

Question 8: Cryptocurrency Market Volatility

Aim

To compare cryptocurrency prices using Cartesian and logarithmic axes.

Problem Statement

Bitcoin, Ethereum, Solana, and Cardano have different price ranges. The objective is to compare their trends using linear and logarithmic scales.

Procedure

1. Import libraries.
2. Generate sample cryptocurrency prices.

3. Plot a linear graph.
4. Plot a logarithmic graph.
5. Compare both graphs.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

days=np.arange(1,31)

btc=np.random.randint(40000,70000,30)
eth=np.random.randint(2000,5000,30)
sol=np.random.randint(50,300,30)
ada=np.random.randint(1,5,30)

plt.plot(days,btc,label="Bitcoin")
plt.plot(days,eth,label="Ethereum")
plt.plot(days,sol,label="Solana")
plt.plot(days,ada,label="Cardano")
plt.legend()
plt.title("Linear Scale")
plt.show()

plt.semilogy(days,btc,label="Bitcoin")
plt.semilogy(days,eth,label="Ethereum")
plt.semilogy(days,sol,label="Solana")
plt.semilogy(days,ada,label="Cardano")
plt.legend()
plt.title("Log Scale")
plt.show()
```

R Code

```
set.seed(20)
```

```
days<-1:30
```

```
btc<-sample(40000:70000,30)
```

```
eth<-sample(2000:5000,30)
```

```
sol<-sample(50:300,30)
```

```
ada<-runif(30,1,5)
```

```
plot(days,btc,  
      type="l",  
      col="red",  
      ylim=c(1,70000),  
      xlab="Days",  
      ylab="Price",  
      main="Linear Scale")
```

```
lines(days,eth,col="blue")
```

```
lines(days,sol,col="green")
```

```
lines(days,ada,col="purple")
```

```
plot(days,btc,  
      type="l",  
      log="y",  
      col="red",  
      ylim=c(1,70000),  
      xlab="Days",  
      ylab="Price",  
      main="Log Scale")
```

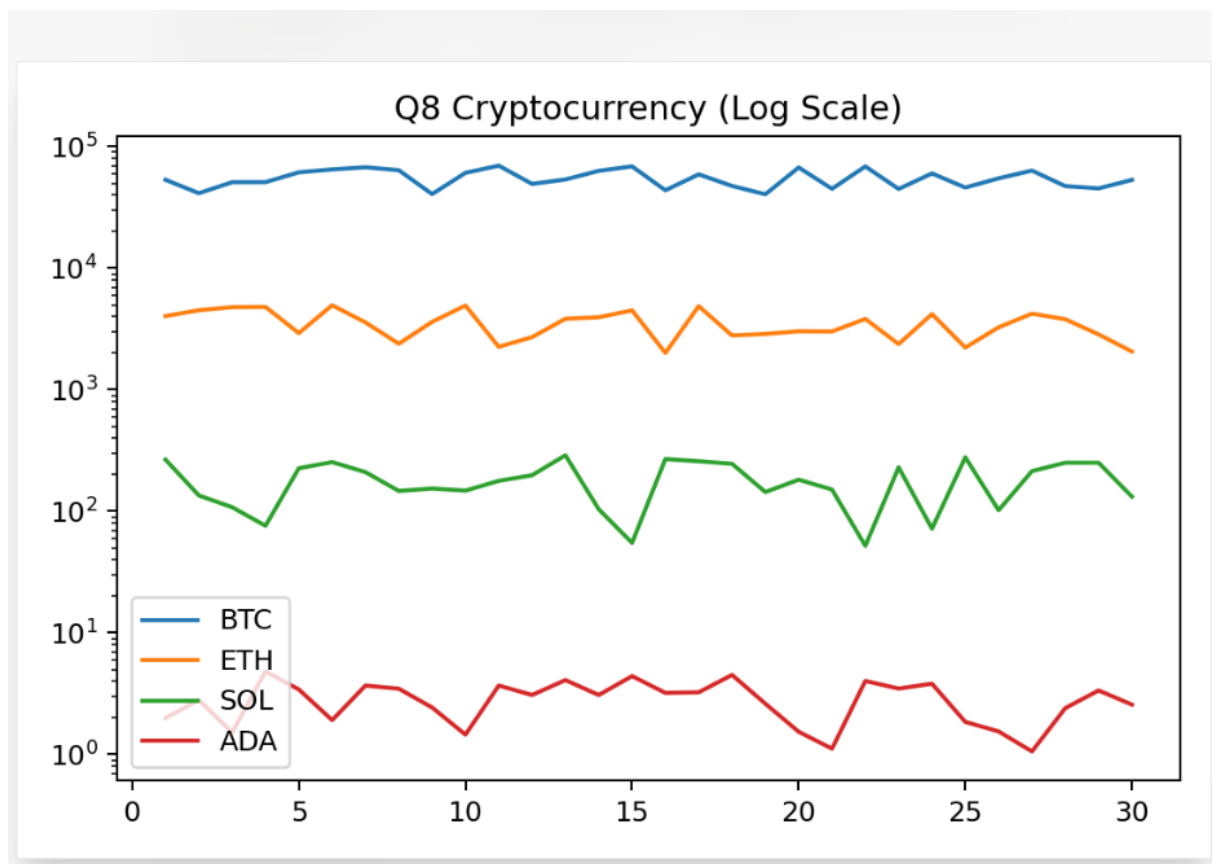
```
lines(days,eth,col="blue")
```

```
lines(days,sol,col="green")
```

```
lines(days,ada,col="purple")
```

Expected Output

- Linear scale cryptocurrency comparison
- Logarithmic scale cryptocurrency comparison
- Outcome:



Result

The logarithmic scale makes it easier to compare cryptocurrencies with vastly different prices by compressing large values while preserving overall trends.

Question 9: Marathon Runner Performance Analysis

Aim

To analyze marathon runner performance using Cartesian and Polar coordinate visualizations with color gradients representing speed.

Problem Statement

A marathon organizer records runner speed, heart rate, distance covered, elevation, and running direction. The objective is to compare Cartesian and Polar plots to identify pacing strategies and directional movement.

Procedure

1. Import the required libraries.
2. Create a sample marathon dataset.
3. Plot speed versus distance using Cartesian coordinates.
4. Plot running direction and speed using Polar coordinates.
5. Compare both visualizations.
6. Analyze pacing strategies and movement direction.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(25)

distance = np.arange(1,51)
speed = np.random.uniform(8,18,50)
direction = np.random.uniform(0,360,50)

# Cartesian Plot
plt.figure(figsize=(7,4))
plt.scatter(distance, speed, c=speed, cmap="viridis", s=60)
plt.colorbar(label="Speed")
plt.xlabel("Distance (km)")
plt.ylabel("Speed (km/h)")
plt.title("Cartesian Plot")
plt.show()

# Polar Plot
theta = np.deg2rad(direction)

plt.figure(figsize=(6,6))
ax = plt.subplot(111, projection="polar")
ax.scatter(theta, speed, c=speed, cmap="plasma", s=60)
```

```
plt.title("Polar Plot")  
plt.show()
```

R Code

```
set.seed(25)
```

```
distance <- 1:50
```

```
speed <- runif(50,8,18)
```

```
direction <- runif(50,0,360)
```

```
# Cartesian Plot
```

```
plot(distance,  
      speed,  
      col=heat.colors(50),  
      pch=19,  
      xlab="Distance (km)",  
      ylab="Speed (km/h)",  
      main="Cartesian Plot")
```

```
# Polar Plot
```

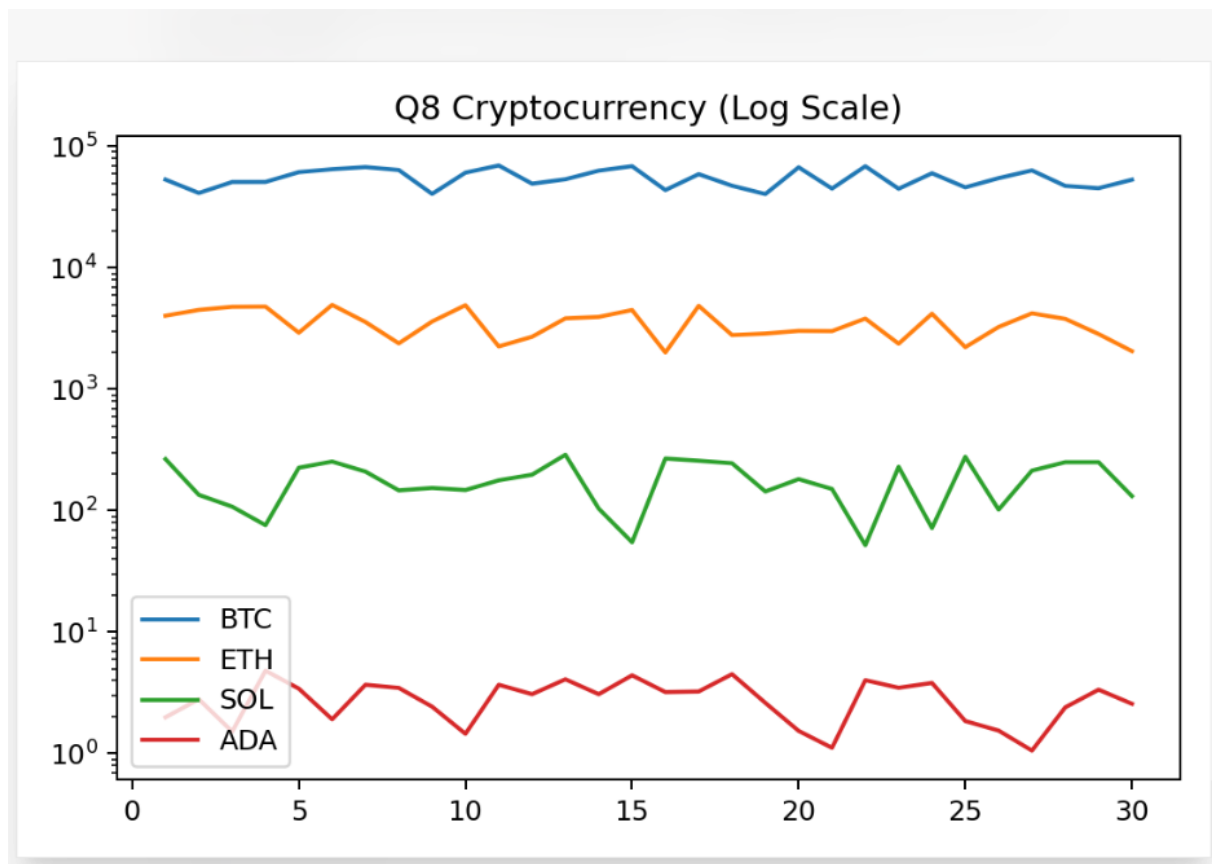
```
theta <- direction*pi/180
```

```
plot(theta,  
      speed,  
      polar=TRUE,  
      col=heat.colors(50),  
      pch=19,  
      main="Polar Plot")
```

Expected Output

- Cartesian plot showing speed versus distance.
- Polar plot showing running direction and speed.

Outcome:



Result

The **Polar Plot** provides better insights into running direction, while the **Cartesian Plot** is useful for analyzing pacing strategies and speed changes over distance.

Question 10: COVID-19 Vaccination Campaign Dashboard

Aim

To design two vaccination dashboards using different color palettes and highlighting strategies for policymakers and the general public.

Problem Statement

Health officials monitor vaccination rates across districts, age groups, and vaccine types. The objective is to create two visualizations that are accessible, easy to interpret, and suitable for users with color vision deficiencies.

Procedure

1. Import the required libraries.
2. Create a sample vaccination dataset.
3. Design a bar chart for policymakers.
4. Design a heatmap for the public.

5. Apply different color palettes.
6. Compare accessibility and interpretability.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

district = ["A","B","C","D","E"]
vaccination = [75,90,65,85,95]

# Bar Chart
plt.figure(figsize=(6,4))
plt.bar(district, vaccination,
        color=["green","blue","orange","purple","red"])
plt.title("Vaccination Dashboard")
plt.xlabel("District")
plt.ylabel("Vaccination (%)")
plt.show()

# Heatmap
data = np.random.randint(50,100,(5,5))

plt.figure(figsize=(5,5))
plt.imshow(data, cmap="YlGnBu")
plt.colorbar(label="Vaccination")
plt.title("Vaccination Heatmap")
plt.show()
```

R Code

```
district <- c("A","B","C","D","E")
vaccination <- c(75,90,65,85,95)
```

```
# Bar Chart
barplot(vaccination,
        names.arg=district,
        col=c("green","blue","orange","purple","red"),
        main="Vaccination Dashboard",
        ylab="Vaccination (%)")
```

```
# Heatmap
set.seed(20)
```

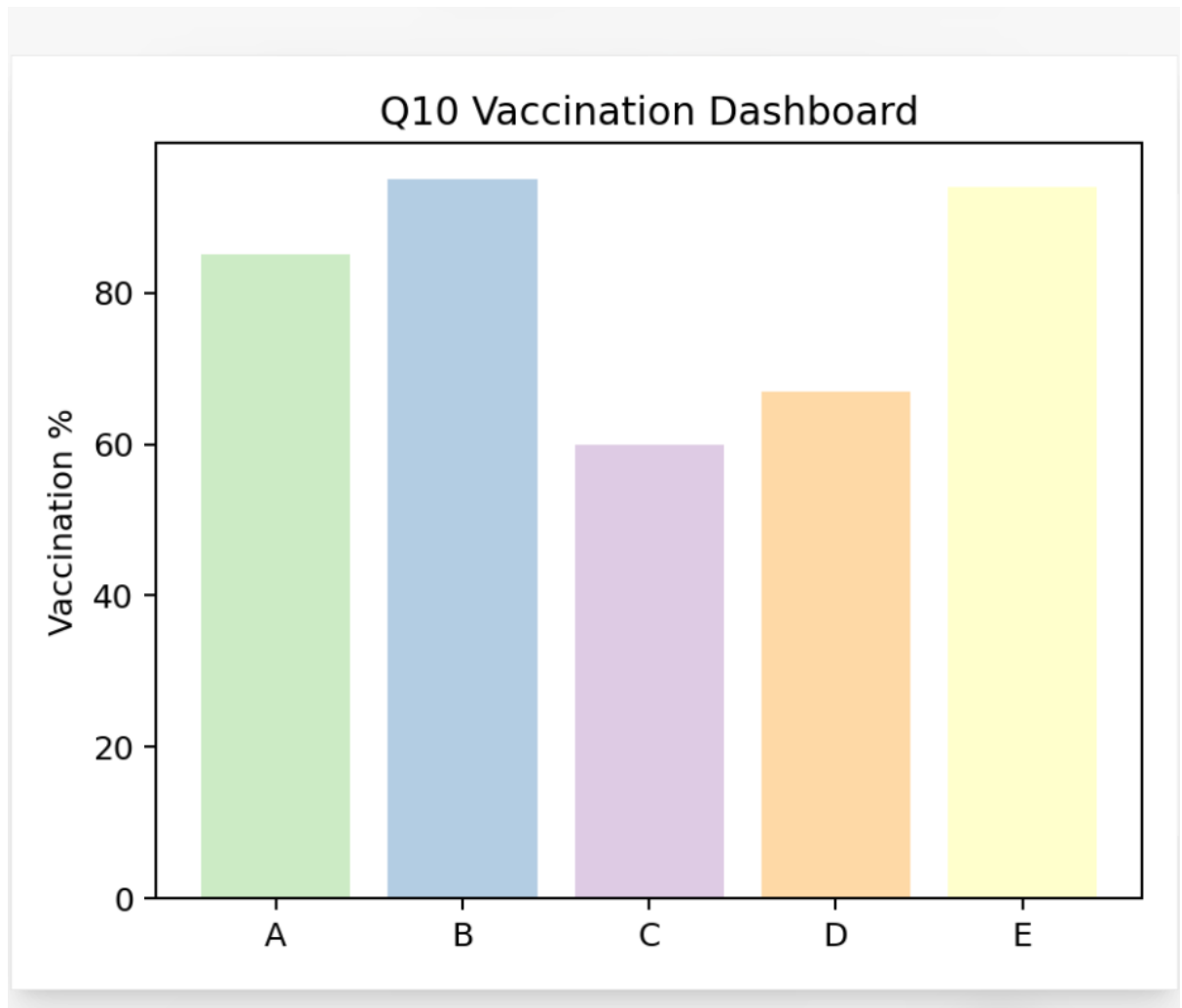
```
data <- matrix(sample(50:100,25),5,5)
```

```
heatmap(data,
        col=terrain.colors(20),
        main="Vaccination Heatmap")
```

Expected Output

- Dashboard 1: Color-coded bar chart showing district-wise vaccination rates.
- Dashboard 2: Heatmap representing vaccination coverage.

Outcome:



Result

The **bar chart** is easy for policymakers to compare vaccination rates across districts, while the **heatmap** provides a quick overview of coverage patterns. Using color-blind-friendly palettes improves accessibility and makes the dashboards easier to interpret for a wider audience.